

Examen-P3-SCD.pdf



DevGenius



Sistemas Concurrentes y Distribuidos



2º Grado en Ingeniería Informática



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada



MÁSTER EN

Inteligencia Artificial & Data Management

MADRID

Formamos
talento para un futuro
Sostenible

saber más



LLÉVATE 2 CLASES PRÁCTICAS GRATIS

Elige entre el permiso A1 o B

APROVECHA LA
SÚPER OFERTA



SISTEMAS CONCURRENTES Y DISTRIBUIDOS

Examen práctica 3 (15/12/2022)

1. Construir un programa MPI, que se deberá llamar **prod_cons_ex.cpp**, operaciones de paso de mensaje síncronas siguiendo el esquema del productor-consumidor, en el que tenemos cuatro procesos consumidores (identificadores de proceso del 0 al 3) y dos procesos productores (identificadores de proceso del 4 al 5) que se comunican mediante un proceso intermedio (identificador 6), que gestiona un vector de tamaño 5. Los productores generarán 30 elementos enteros cada uno. Otras consideraciones:

- La gestión del vector se debe hacer en modo FIFO.
- Cuando el proceso intermedio recibe un elemento concreto de cada productor, no puede atender peticiones de ningún productor hasta vaciar el vector. Es decir, cuando el proceso intermedio reciba el décimo elemento producido por el productor con identificador 5 (sean cuales sean los valores), debe indicar que se ha dado esa situación con un mensaje por pantalla y no puede recibir elementos de ningún productor hasta que no haya enviado todos los elementos que haya en ese momento en el vector a los procesos consumidores.

2. En la tienda oficial de Catar hay 2 taquillas para comprar entradas (denominadas 'A' y 'B') y a ella entran a comprar 12 'forofos' del fútbol (procesos 0 al 11). Los procesos forofos repiten un bucle de compra de entradas y resto de código. Para gestionar el pago, se utiliza un sistema de cola única que gestiona un proceso encargado (proceso 12) que recibe peticiones de pago de los forofos y controla la ocupación de las taquillas para dar paso a dichos forofos (**las taquillas no son procesos, solo es necesario que el proceso encargado sepa si una está libre u ocupada**). Si hay al menos una taquilla libre, y recibe la petición de un forofo, le envía en la variable del mensaje la taquilla asignada (si ambas están libres, se asigna la B). Cuando el forofo finaliza la compra, manda un mensaje al proceso encargado para avisar que se ha quedado libre la taquilla asignada. Construir un programa MPI, que se deberá llamar **ejer2p3.cpp**, con operaciones de paso de mensajes síncronas y que cumpla las características descritas de acuerdo al siguiente esquema (todos los procesos son bucles infinitos).

Proceso funcion_forofo_futbolero

{retardo aleatorio} (decidir qué entrada compra)

Enviar un mensaje al proceso encargado {solicitar taquilla}

Recibir un mensaje del proceso encargado {ya puede ir a la taquilla indicada en la variable del mensaje}

Imprime texto por pantalla: "soy el forofo X y me ha tocado la taquilla Y"

{retardo aleatorio} (compra de la entrada)

Enviar un mensaje al proceso encargado con la taquilla asignada en la variable del mensaje {compra completada}

{retardo aleatorio} (resto código del forofo dando botes de alegría porque ha conseguido una entrada)

Proceso funcion_encargado

Si hay alguna taquilla libre

el mensaje a recibir puede ser de solicitar taquilla o de compra completada

En caso contrario (no hay taquillas libres)

el mensaje a recibir solo puede ser de compra completada

Recibir mensaje

Si se ha recibido un mensaje de solicitar taquilla

actualizar estado taquillas (una taquilla libre menos)

taq:= seleccionar una taquilla libre

la taquilla de la variable "taq" pasa a estar ocupada

enviar mensaje al proceso forofo {se le envía la variable "taq" para que el forofo

sepa la taquilla asignada}

Si se ha recibido un mensaje de compra completada

actualizar estado taquillas (una taquilla libre más)

la taquilla de la variable recibida pasa a estar libre

ESCANÉA EL QR
Llévate 2 clases prácticas
totalmente **Gratis**

